

Extreme Value Analysis (EVA) Platform — Complete Implementation Blueprint

1. Project Overview

Objective

Build a production-grade Extreme Value Analysis (EVA) platform for:

- Corrosion prediction
- Wall-loss forecasting
- Risk analysis
- Inspection planning
- Remaining life estimation
- Reliability engineering
- Extreme event prediction

The system should:

- Accept inspection and wall-loss datasets
- Fit statistical extreme value distributions
- Estimate return levels
- Generate confidence intervals
- Validate goodness-of-fit
- Produce engineering reports and visualizations
- Support multi-tenant SaaS deployment

2. Recommended Tech Stack

Layer	Technology
Frontend	Next.js + TypeScript
UI	Tailwind + Shadcn UI
Backend API	NestJS / Express + TS
Statistical Engine	Python FastAPI
Statistical Libraries	NumPy, SciPy, Pandas
Database	PostgreSQL
Queue	BullMQ + Redis

Layer	Technology
Storage	AWS S3
Deployment	AWS ECS
Charts	Plotly
Monitoring	Grafana + Sentry

3. System Architecture

```

Frontend (Next.js)
  ↓
API Gateway (NestJS)
  ↓
Analysis Queue (BullMQ)
  ↓
Python EVA Engine (FastAPI)
  ↓
PostgreSQL + S3

```

4. Complete Application Flow

Step 1 — User Authentication

Features

- Login
- Registration
- Role-based access control
- Tenant isolation
- JWT authentication

Roles

Role	Permissions
Admin	Full access
Engineer	Run EVA analysis
Viewer	Read-only

5. Main Modules

Module	Purpose
Authentication	Login & security
Asset Management	Manage equipment/assets
Dataset Upload	Upload inspection data
EVA Analysis	Statistical calculations
Visualization	Probability & return plots
Reporting	PDF/export generation
Notifications	Risk alerts
Dashboard	Monitoring & KPIs

6. Frontend Page Structure

6.1 Login Page

Purpose

Authenticate users.

Inputs

Field	Type
Email	Text
Password	Password

Outputs

- JWT token
 - Session creation
-

6.2 Dashboard Page

Purpose

Display summary statistics.

Components

- Total assets
- Total inspections
- Risk overview
- Latest EVA runs
- High-risk alerts
- Return-level summaries

Charts

- Risk trend chart
 - Failure probability chart
 - Asset condition chart
-

6.3 Asset Management Page

Purpose

Create and manage industrial assets.

Inputs

Field	Type
Asset Name	Text
Asset Type	Dropdown
Material	Dropdown
Location	Text
Installation Date	Date
Design Thickness	Number
Corrosion Allowance	Number

Outputs

- Asset record
 - Asset metadata
-

6.4 Dataset Upload Page

Purpose

Upload wall-loss and inspection datasets.

Supported Formats

- CSV
- Excel (.xlsx)

Inputs

Field	Type
Asset Selection	Dropdown
Upload File	File
Measurement Unit	Dropdown
Date Format	Dropdown

Required Dataset Columns

Column	Description
inspection_date	Date of inspection
wall_loss	Measured wall loss
thickness	Remaining thickness
location	Measurement location
sensor_id	Optional sensor identifier

Validation Rules

- No negative wall-loss
- No missing dates

- Numeric validation
- Duplicate check
- Unit consistency

Outputs

- Dataset ID
 - Validation status
 - Parsed records
-

6.5 EVA Configuration Page

Purpose

Configure EVA analysis.

Inputs

Field	Type
Distribution Type	Dropdown
Estimation Method	Dropdown
Confidence Level	Dropdown
Return Periods	Multi-select
Peak Extraction Method	Dropdown
Outlier Handling	Toggle

Distribution Options

- Gumbel
- GEV
- Weibull
- Generalized Pareto (future)

Estimation Methods

- Maximum Likelihood Estimation (MLE)
- Method of Moments (MoM)

Peak Extraction Methods

- Annual maxima
- Peak over threshold (future)

Outputs

- EVA configuration object
-

6.6 EVA Results Page

Purpose

Display statistical outputs.

Sections

1. Distribution parameters
 2. Return levels
 3. Goodness-of-fit
 4. Confidence intervals
 5. Probability plots
 6. Diagnostics
 7. Engineering recommendations
-

7. EVA Analysis Engine

7.1 Input Data Structure

Required Input JSON

```
{
  "datasetId": "123",
  "distribution": "gumbel",
  "method": "mle",
  "confidenceLevel": 0.95,
  "returnPeriods": [2, 5, 10, 25, 50, 100]
}
```

7.2 Preprocessing Stage

Operations

1. Remove invalid rows
2. Remove NaN values
3. Sort descending
4. Extract maxima
5. Normalize units
6. Validate sample size

Formula

Number of observations:

$$n = \text{length of dataset}$$

7.3 Statistical Formulas

Gumbel Distribution PDF

$$f(x; \mu, \beta) = \frac{1}{\beta} \exp\left(-\frac{x - \mu}{\beta}\right) \exp\left(-e^{-(x - \mu)/\beta}\right)$$

Gumbel Distribution CDF

$$F(x) = \exp\left(-e^{-(x - \mu)/\beta}\right)$$

Method of Moments Initialization

Scale Parameter

$$\beta = \sigma\sqrt{6}/\pi$$

Location Parameter

$$\mu = \bar{x} - \gamma\beta$$

Where:

Symbol	Meaning
σ	Standard deviation
$\bar{x}$	Mean
γ	Euler constant

Maximum Likelihood Estimation (MLE)

Negative Log Likelihood

$$L(\mu, \beta) = n \ln(\beta) + \sum z_i + \sum e^{-z_i}$$

Where:

$$z_i = \frac{x_i - \mu}{\beta}$$

Optimization Method

Recommended:

- L-BFGS-B
- Nelder-Mead

Return Level Formula

Reduced variate:

$$y_T = -\ln \left(-\ln \left(1 - \frac{1}{T} \right) \right)$$

Return level:

$$x_T = \mu + \beta y_T$$

Where:

Symbol	Meaning
T	Return period
x_T	Return level

Probability Plotting Position

$$P_i = \frac{i}{n+1}$$

Reduced variate:

$$y_i = -\ln(-\ln(P_i))$$

Anderson-Darling Test

AD Statistic

$$A^2 = -n - \frac{1}{n} \sum_{i=1}^n (2i-1) [\ln(F(x_i)) + \ln(1-F(x_{n+1-i}))]$$

Purpose

Measure goodness-of-fit.

Confidence Intervals

Methods

1. Hessian matrix
2. Fisher Information Matrix
3. Bootstrap resampling

Recommended Production Method

Bootstrap.

8. EVA Processing Pipeline

Step-by-Step Flow

```
Upload dataset
  ↓
Validate data
  ↓
Extract maxima
  ↓
Initialize parameters (MoM)
  ↓
Run MLE optimization
  ↓
Compute return levels
  ↓
Compute confidence intervals
  ↓
Run AD test
  ↓
Generate plots
  ↓
Store results
  ↓
Generate report
```

9. Backend API Design

Upload Dataset

Endpoint

```
POST /datasets/upload
```

Request

Multipart file upload.

Response

```
{
  "datasetId": "abc123",
  "status": "validated"
}
```

Run EVA Analysis

Endpoint

```
POST /eva/run
```

Request

```
{
  "datasetId": "abc123",
  "distribution": "gumbel",
  "method": "mle"
}
```

Response

```
{
  "runId": "eva_001",
  "status": "processing"
}
```

Fetch EVA Results

Endpoint

```
GET /eva/:id/results
```

10. Database Design

users

Column	Type
id	UUID
tenant_id	UUID
email	Text
role	Text

assets

Column	Type
id	UUID
tenant_id	UUID
name	Text
material	Text
location	Text
design_thickness	Float

inspections

Column	Type
id	UUID
asset_id	UUID
inspection_date	Date
wall_loss	Float
thickness	Float

eva_runs

Column	Type
id	UUID
asset_id	UUID
distribution	Text
mu	Float
beta	Float
ad_score	Float
created_at	Timestamp

return_levels

Column	Type
id	UUID
eva_run_id	UUID
return_period	Integer
predicted_value	Float
ci_lower	Float
ci_upper	Float

11. Required Visualizations

Probability Plot

Purpose

Compare observed vs theoretical values.

Return Level Plot

X-axis

Return period.

Y-axis

Extreme value.

Confidence Interval Plot

Purpose

Display uncertainty bands.

Residual Plot

Purpose

Check model quality.

12. Report Generation

Report Sections

1. Asset details
2. Dataset summary
3. Distribution used
4. Estimated parameters
5. Return levels
6. Confidence intervals
7. AD test results
8. Diagnostic plots
9. Engineering interpretation
10. Inspection recommendation

Export Formats

- PDF
 - CSV
 - Excel
-

13. Queue Architecture

Why Needed

EVA calculations are CPU intensive.

Flow

```
Frontend
  ↓
API Gateway
  ↓
BullMQ Queue
  ↓
Python Worker
  ↓
Store results
```

14. Python Statistical Engine Structure

```
eva-engine/
├─ api/
├─ services/
├─ statistics/
├─ optimization/
├─ diagnostics/
├─ plotting/
├─ bootstrap/
├─ reports/
└─ utils/
```

15. Core Python Modules

preprocessing.py

Responsibilities:

- data cleaning
 - sorting
 - validation
 - maxima extraction
-

distributions.py

Responsibilities:

- Gumbel PDF
 - Gumbel CDF
 - GEV support
-

mle.py

Responsibilities:

- negative log likelihood
 - parameter optimization
 - convergence checking
-

diagnostics.py

Responsibilities:

- AD test
 - KS test
 - residual analysis
-

bootstrap.py

Responsibilities:

- confidence intervals
 - uncertainty estimation
-

plotting.py

Responsibilities:

- return plots
 - QQ plots
 - PP plots
 - CI visualization
-

16. Numerical Stability Requirements

Critical Safeguards

Use:

```
np.clip()  
log-sum-exp  
parameter bounds  
small epsilon constants
```

Prevent

- overflow
 - underflow
 - NaN errors
 - optimizer divergence
-

17. Production Recommendations

Mandatory

- Use Python for statistics
 - Use PostgreSQL
 - Use asynchronous processing
 - Add bootstrap confidence intervals
 - Add convergence diagnostics
 - Add logging
 - Add audit trails
-

Avoid

- Running heavy math in frontend
 - Using Excel as production engine
 - Using raw JavaScript math for MLE
 - Ignoring uncertainty estimation
-

18. Future Enhancements

Statistical

- Full GEV support
 - Bayesian EVA
 - POT analysis
 - Automatic distribution selection
 - Time-dependent EVA
-

AI Features

- Risk prediction AI
 - Inspection recommendation AI
 - Failure forecasting
 - Automated anomaly detection
-

Enterprise Features

- Multi-tenancy
 - Team collaboration
 - API access
 - Audit logging
 - Asset hierarchy
-

19. MVP Development Order

Phase 1

Build:

- Authentication
 - Dataset upload
 - Basic Gumbel EVA
 - Return levels
 - Probability plots
-

Phase 2

Add:

- Bootstrap CI
 - AD test
 - Reporting engine
 - Dashboard analytics
-

Phase 3

Add:

- Multi-tenancy
 - RBAC
 - Notifications
 - Asset management
-

Phase 4

Add:

- GEV
 - Bayesian analysis
 - AI forecasting
 - Advanced diagnostics
-

20. Final Recommended Architecture

```
Next.js Frontend
  ↓
NestJS API Gateway
  ↓
Redis Queue
  ↓
Python FastAPI EVA Engine
  ↓
PostgreSQL + S3
```

21. Final Goal

The final system should:

- Accept industrial inspection datasets
- Perform statistically valid EVA analysis
- Estimate extreme future wall loss
- Quantify uncertainty
- Recommend inspection intervals
- Generate engineering-grade reports
- Scale as a multi-tenant SaaS platform
- Support enterprise reliability engineering workflows